NATIONAL INSTITUTE OF TECHNOLOGY DURGAPUR

FINAL YEAR PROJECT REPORT DOCUMENT

NAME: PRIYANKA TAMANG

ROLL: 22CS8067

FINAL YEAR PROJECT ON "STRESS DETECTION USING MACHINE LEARNING"

1 — Project Objective

The primary objective of this project is:

"To compare multiple machine learning classification algorithms and identify the most accurate model and the key features influencing stress prediction."

The system analyzes keystroke dynamics, user behavioral metrics, and psychometric indicators to determine whether a user is stressed or not.

---

2 — Workflow Diagram

The complete workflow followed in the project is shown below:

GET DATASET

↓

PRE-PROCESS DATASET

↓

APPLY TIME SERIES ANALYSIS

↓

APPLY DIFFERENT CLASSIFICATION METHODS

↓

GET ACCURACY & TOP 5 POSITIVE FEATURES

↓

REPRESENT ACCURACY IN GRAPH

↓

FIND COMMON IMPORTANT FEATURES

↓

USE THOSE FEATURES ON SIMILAR DATASETS

---

3 — What We Did

Importing Required Libraries

The project was implemented on **Google Colab** using Python 3.10 .

The following libraries were used:

- **pandas** — for data manipulation
- **numpy** — for numerical operations
- **matplotlib / seaborn** — for visualization
- **scikit-learn** — for model building and evaluation

Data Loading and Exploration

- The dataset was originally available in **TSV** format on Kaggle.
- Converted to **XLS / CSV** for easier handling in Python.
- Initial data exploration was performed using:

df.head() df.info()

df.describe()

This helped understand distribution, feature types, and missing values.

---

4 — Feature Engineering

To improve model performance, several new features were generated from keystroke behavior.

Time-Based Features

- **Key_Duration_s**
  Time difference between pressing and releasing a key.
- **Time_Between_Keystrokes_s**
  Time gap between two consecutive keystrokes.

Key Classification Features

Each keystroke was mapped into one of four groups:

- **Action Keys**
- **Modifier Keys**
- **Navigation Keys**
- **Typing Keys**

```python
def classify_key(key):
    if key in ['enter', 'delete', 'backspace']:
        return 'Action'
    elif key in ['shift', 'ctrl', 'alt_l', 'caps_lock']:
        return 'Modifier'
    elif key == 'arrow_key':
        return 'Navigation'
    else:
        return 'Typing'
```

•

Psychological & Behavioral Encodings User

states such as:

- **Fatigue**
- **Energy Level**
- **Pleasantness**
- **Stress Mapping** were all encoded numerically:

```python
fatigue_mapping = {'Low': 0, 'Below_Avg': 1, 'Avg': 2, 'Above_Avg': 3, 'V_High': 4, 'No': 0}
stress_mapping = {'Neutral': 0, 'F_Good': 0, 'F_Great': 0, 'S_Stressed': 1, 'V_Stressed': 2, 'No': 0}
energy_mapping = {'Neutral': 0, 'S_Low_Energy': 1, 'V_Low_Energy': 2, 'S_Energetic': 3, 'No': 0}
pleasant_mapping = {'Neutral': 0, 'S_Pleasant': 1, 'S_Unpleasant': -1, 'V_Unpleasant': -2, 'No': 0}
```

---

5 — Model Training & Classification Analysis

The input X and output y were prepared as: feature_columns

= [

'Key_Duration_s', 'Time_Between_Keystrokes_s', 'Fatigue_Num', 'PAM_Val',

'Energy_Num', 'Pleasant_Num', 'Daylight_user_Afternoon',

'Daylight_user_Evening', 'Daylight_user_Morning',

'Daylight_key_Afternoon', 'Daylight_key_Evening', 'Daylight_key_Morning',

'KeyType_Action', 'KeyType_Modifier', 'KeyType_Navigation', 'KeyType_Typing'

]

X = combined_df[feature_columns] y

= combined_df['Is_Stressed']

Train-Test Split

- **70%** data → Training
- **30%** data → Testing

Algorithms Used

The following classification models were applied:

- **Logistic Regression**
- **Random Forest**
- **XG Boost**
- **Gradient Boosting**
- **Linear Regression**

Each model's accuracy and feature importance values were extracted and recorded.

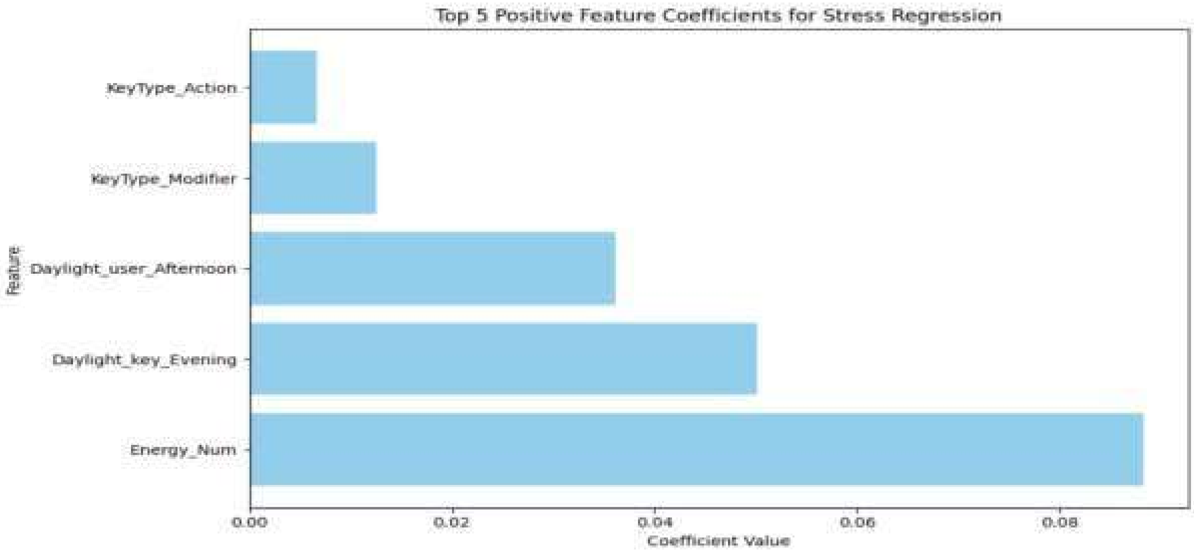

Fig 1: Linear Regression

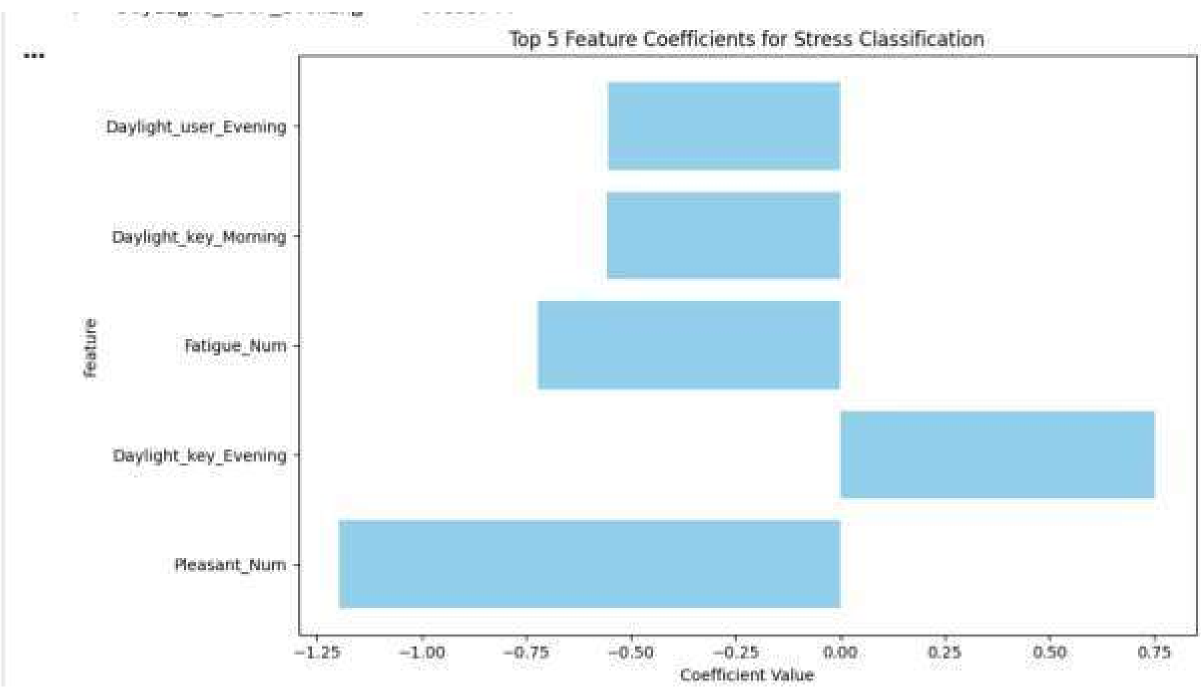

Fig 2: Logistic Regression

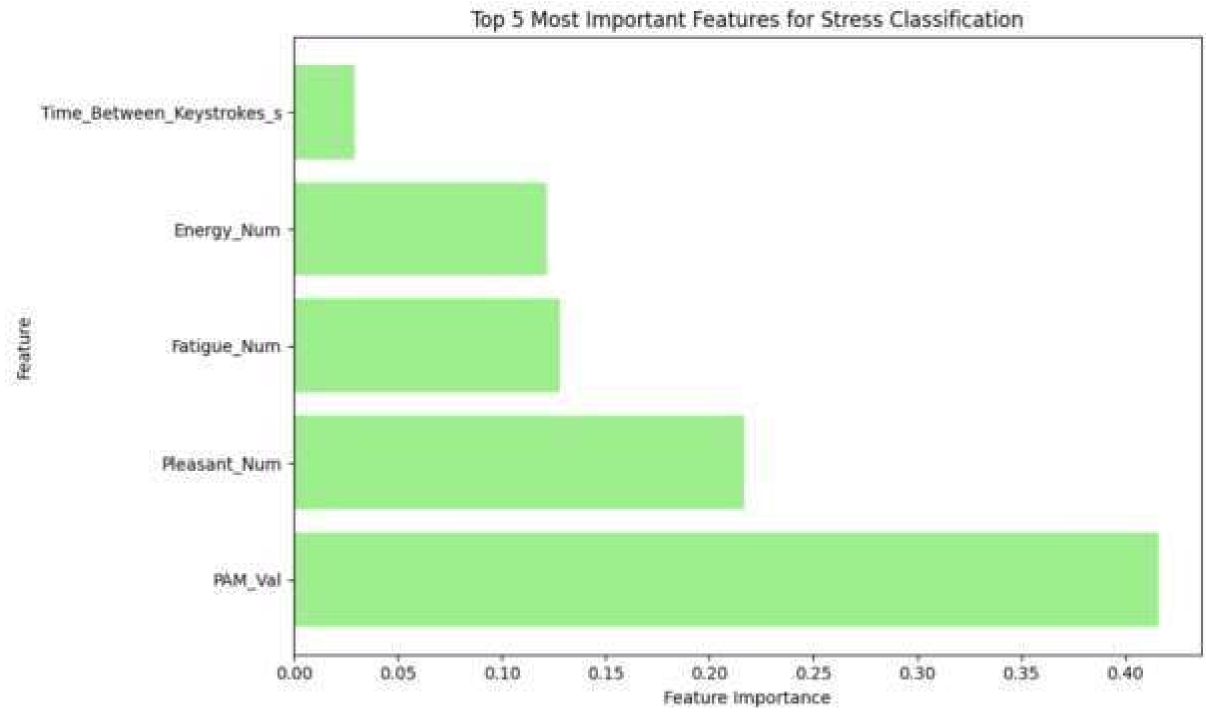

Fig 3: Random Forest

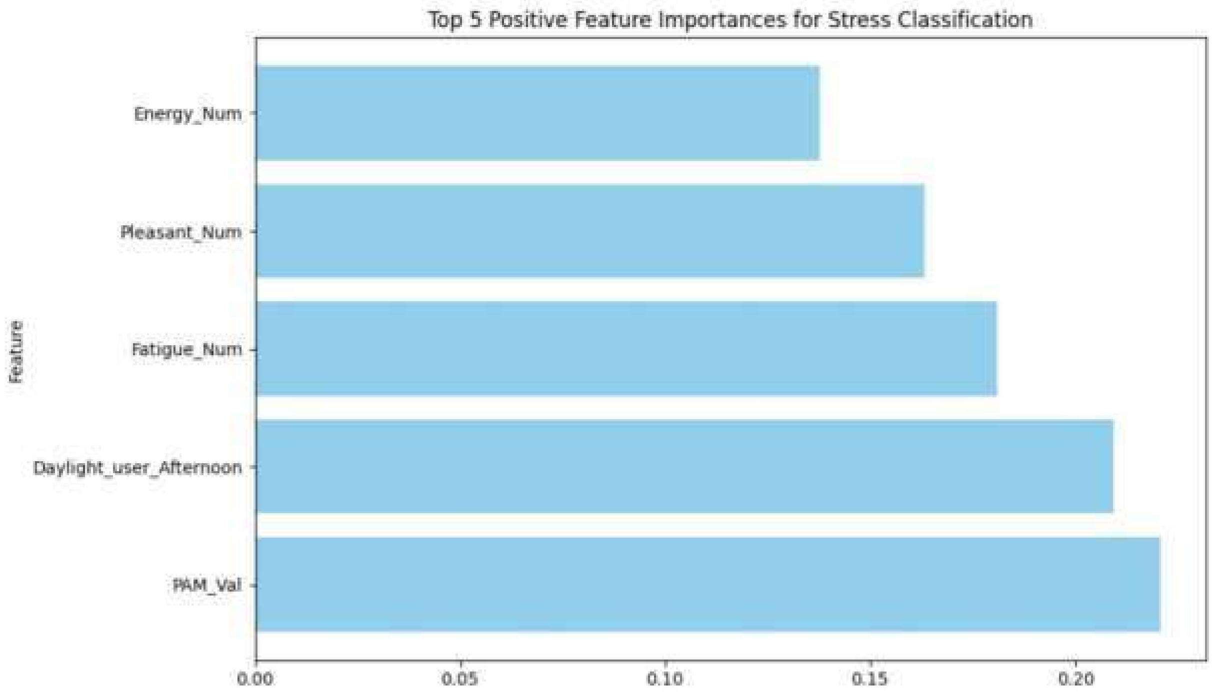

Fig 4: XG Boost

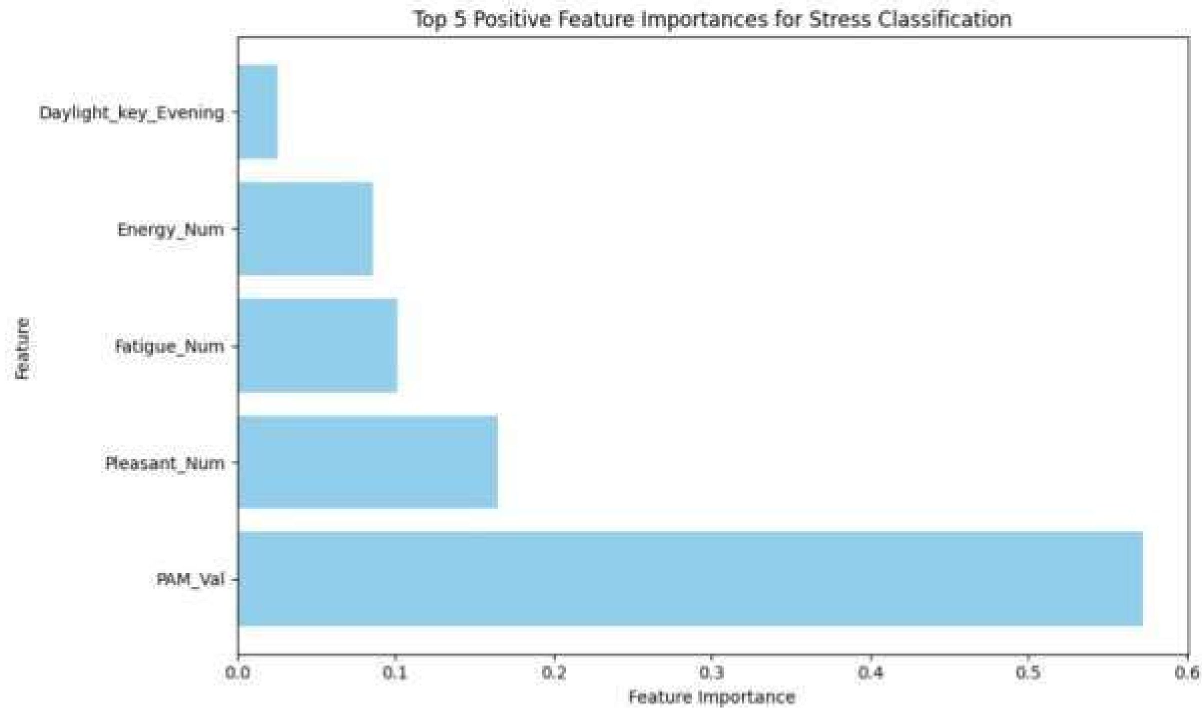

Fig 5: Gradient

6 — Results & Observations

Accuracy Comparison

A visualization graph was generated to show differences in accuracy among classifiers.

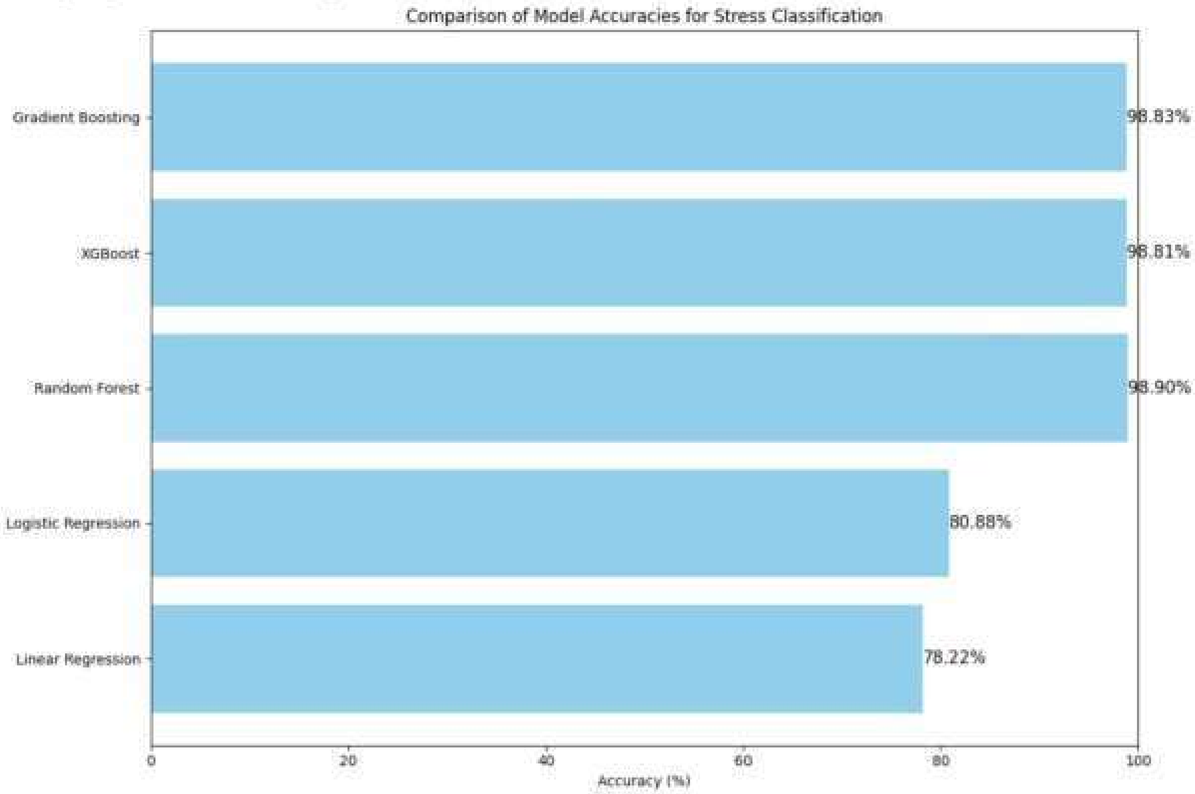

Fig 6: Comparison of Model Accuracies for Stress Classification

- **Random Forest** delivered the **highest accuracy** among all algorithms.

Top 5 Important Features

For each classifier, the top five positive contributing features were extracted.

Example features with strong impact:

- Energy_Num
- Key_Duration_s
- Time_Between_Keystrokes_s
- Pleasant_Num
- Fatigue_Num

Common Features Across All Classifiers

The commonly occurring influential features were:

Energy_Num (most consistent)
Key timing features (duration & interval) Daylight-based context features

These features strongly correlate with the user's stress level.

---

7 — Extended Evaluation

After identifying the top features:

- The reduced feature set was applied to another similar dataset.
- The model maintained strong accuracy with fewer features.
- This improves model performance and reduces training complexity.

---

8 — Conclusion

Based on the experimental results:

1. **Random Forest Classifier** achieved the highest accuracy.
2. **Energy_Num** emerged as the most important feature across all models.
3. Keystroke dynamics combined with psychological measures provide a strong indicator of user stress.
4. Feature reduction improves generalization and speeds up computation.

---

9 — Real-World Applications

This stress detection system can be used for:

Mental Health Monitoring

Detects stress early without wearables or surveys.

Workplace Wellness Tracking

Organizations can anonymously track worker stress patterns and recommend breaks.

Personal Wellness Apps

Can integrate into chat apps, typing platforms, and mobile apps for real-time stress alerts.

Human–Computer Interaction Research

Supports Affective Computing — enabling systems to understand human emotions from typing behavior.

---

10 — Future Work

- Apply **deep learning models** (LSTM / GRU) for sequential keystroke patterns
- Collect **real-time keystroke data**
- Build a **live dashboard** for stress monitoring
- Use **common influential features** to create lightweight, real-time models.

PROJECT LINK: stress detection using ML

*****The End*****
